

MAT31 - 92

Applied Probability and Simulation

[Module page](#)

Version of 13 September 2026

About this textbook

This year, for the first time, every Bachelor course at Albert School has its own textbook, thanks to your teachers and AI.

- The textbook is a help for working on the course at home, after the class.
- The course as taught in class remains the reference, and prevails over the textbook wherever the two differ.
- This is a first version. Help us improve it by sharing your feedback with your teacher.

Contents

Preface	4
1. Simulating the discrete laws	5
1.1. The generator, and one line per law	5
1.2. Reading the shape, not just the summaries	6
2. Simulating the continuous laws	8
2.1. From samples to a density	8
2.2. Why the tail matters	10
2.3. Where the samples come from: inverting the CDF	10
3. The Law of Large Numbers and the Central Limit Theorem	13
3.1. The Law of Large Numbers, watched live	13
3.2. The Central Limit Theorem, and the size of the error	14
4. Maximum-likelihood estimation	17
4.1. Likelihood and log-likelihood	17
4.2. The three closed-form estimators	17
5. Checking an estimator by simulation	20
5.1. The repeated-sampling experiment	20
6. Chi-square tests	22
6.1. The chi-square statistic	22
6.2. Goodness of fit	22
6.3. Independence of two variables	23
7. Comparing a mean or a proportion	25
7.1. Testing against a benchmark, and between two groups	25
7.2. Where the two-proportion statistic comes from	27
7.3. How large a sample?	27
8. Resampling methods	30
8.1. The bootstrap: resample with replacement	30
8.2. The permutation test: reshuffle without replacement	30
8.3. Which to reach for	31
Primary sources	32

Preface

Your previous probability course gave you the classic laws, the Law of Large Numbers, the Central Limit Theorem, confidence intervals and hypothesis tests as results you computed by hand — an integral, a table lookup, a formula. This course puts the computer in that place: it generates the randomness and you read the answer off the sample. By the end you will be able to

- simulate each classic discrete law (uniform, Bernoulli, geometric, binomial, Poisson) and each classic continuous law (uniform, exponential, normal, Pareto) in Python, and read a law's behaviour back from its sample;
- watch the Law of Large Numbers and the Central Limit Theorem emerge from simulated samples, and put a number on the sampling error the CLT governs;
- derive, by maximising the log-likelihood, the closed-form maximum-likelihood estimators of the Bernoulli, Poisson and exponential parameters, and check an estimator's consistency, variance decay and asymptotic normality by simulation;
- run a chi-square test — for goodness of fit, or for the independence of two categorical variables — and a z- or t-test comparing a mean or a proportion against a benchmark or between two groups, and size an experiment before running it;
- prove the CDF Inversion Theorem, the one construction behind every sampler in the book;
- recognise the two resampling ideas, the bootstrap and the permutation test, and say when each applies.

Two things this book leans on but never reteaches: fluency with short Python written in `numpy` and `matplotlib`, and the probability your earlier course established — the classic laws, the Law of Large Numbers and the Central Limit Theorem, confidence intervals and hypothesis tests. Bring both; every chapter puts them to work, and none rebuilds them.

One discipline runs through all of it: *an estimate is not an answer until it carries its uncertainty*. Producing that uncertainty alongside the estimate, by simulation, is the craft this course teaches.

1. Simulating the discrete laws

Every simulation in this book rests on one ability: turning a named distribution into a stream of sample values. This chapter builds that ability for the classic *discrete* laws you already know analytically — the uniform, Bernoulli, geometric, binomial and Poisson — and shows how to *read a law's behaviour back from its samples*, checking the simulation against the theory you have already met.

1.1. The generator, and one line per law

All randomness in this course flows from NumPy's `numpy.random` module, which we alias `rd`. We seed it once, so the whole book is reproducible: run it again and every number returns identical.

```
import numpy as np
import numpy.random as rd

rd.seed(20260730)
```

From `rd`, each classic discrete law is a single call. The table below draws a large sample from each and reports two summaries — the *sample mean* and *sample variance* — beside the *theoretical* mean and variance you can compute from the law's formula. The Law of Large Numbers (Chapter 3) guarantees these must nearly agree once the sample is large.

```
# Each classic discrete law has a one-line sampler on rd. Draw a large sample
# from each, then read its behaviour back from the sample: the sample mean and
# variance should sit close to the law's theoretical mean and variance.
n = 100000

samples = {
    "Uniform{1..6}": rd.randint(1, 7, size=n),          # a fair die
    "Bernoulli(0.3)": rd.binomial(n=1, p=0.3, size=n),  # one trial
    "Geometric(0.25)": rd.geometric(0.25, size=n),      # trials to 1st success
    "Binomial(10, 0.3)": rd.binomial(n=10, p=0.3, size=n), # successes in 10 trials
    "Poisson(4)": rd.poisson(4, size=n),                # events per interval
}

theory = {                                              # (mean, variance)
    "Uniform{1..6}": (3.5, 35 / 12),
    "Bernoulli(0.3)": (0.3, 0.3 * 0.7),
    "Geometric(0.25)": (1 / 0.25, (1 - 0.25) / 0.25 ** 2),
    "Binomial(10, 0.3)": (10 * 0.3, 10 * 0.3 * 0.7),
    "Poisson(4)": (4, 4),
}

head = "{:<18}{:>12}{:>12}{:>12}{:>12}"
row = "{:<18}{:>12.3f}{:>12.3f}{:>12.3f}{:>12.3f}"
print(head.format("law", "mean (sim)", "mean (thy)", "var (sim)", "var (thy)"))
for name in samples:
    x = samples[name]
    m_thy, v_thy = theory[name]
    print(row.format(name, np.mean(x), m_thy, np.var(x, ddof=1), v_thy))
```

law	mean (sim)	mean (thy)	var (sim)	var (thy)
Uniform{1..6}	3.498	3.500	2.922	2.917
Bernoulli(0.3)	0.299	0.300	0.210	0.210
Geometric(0.25)	3.983	4.000	11.967	12.000
Binomial(10, 0.3)	3.000	3.000	2.091	2.100
Poisson(4)	3.994	4.000	4.002	4.000

Every simulated mean and variance sits within a percent or two of its theoretical value: the samplers are faithful. Reading the columns across is the skill this chapter trains — *the sample is a window onto the law*, and its summaries recover the law’s parameters.

Method 1 (Simulating a discrete law and checking it).

1. Pick the `rd` sampler for the law and its parameters (e.g. `rd.poisson(4, size=n)`).
2. Draw a *large* sample of size n (here $n = 100\,000$).
3. Compute sample summaries — `np.mean(x)`, `np.var(x, ddof=1)`, or the fraction of draws equal to each value — and compare them to the law’s theoretical mean, variance, or pmf.
4. Large gaps signal a wrong sampler, wrong parameters, or too small an n .

Remark. The samplers use conventions worth pinning down. `rd.randint(1, 7, size=n)` draws integers from 1 to 6 (the upper end is *excluded*) — a fair die. `rd.geometric(p, size=n)` counts the *number of trials up to and including the first success*, so its support starts at 1 and its mean is $1/p$; a different convention (failures *before* the first success) would start at 0. Always confirm which convention a function uses before trusting its output.

Remark. NumPy also offers a newer *generator* interface, whose `rng.integers(low, high, size)` is this course’s `rd.randint(low, high, size)`, both excluding the upper bound.

1.2. Reading the shape, not just the summaries

Mean and variance are two numbers; the *whole* law is its probability mass function, the height at each possible value. Overlaying the *empirical frequencies* (the fraction of draws landing on each value) on the *theoretical pmf* is the sharpest check of all.

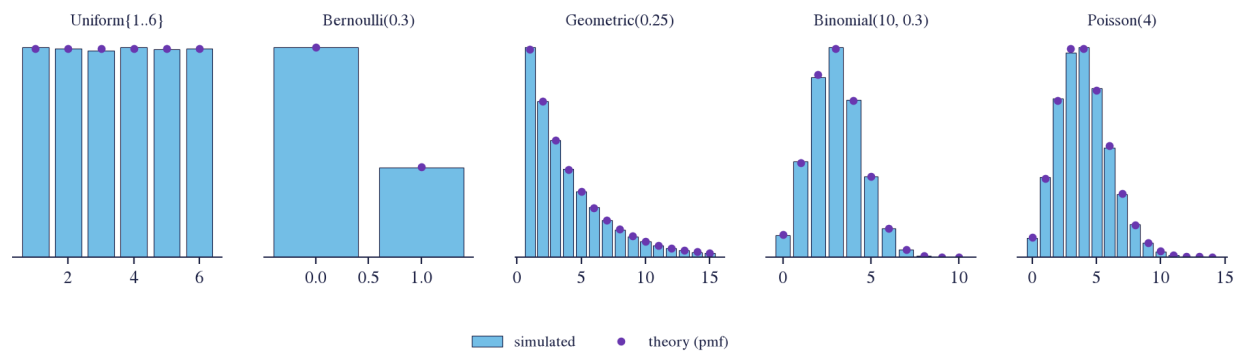


Figure 1: Each classic discrete law, simulated. Bars are the empirical frequencies from 100 000 draws; purple dots are the theoretical pmf. Left to right: the *uniform* die is flat; *Bernoulli* puts mass 0.3 on 1 and 0.7 on 0; the *geometric* law decays geometrically from its mode at 1; the *binomial* is a bump centred near $np = 3$; the *Poisson* is the right-skewed count law with mean 4. Bars track dots everywhere — the simulation reproduces each law’s shape.

The pictures make each law’s characteristic shape easy to read: the geometric law’s steady decay (each extra trial is less likely than the last), the binomial’s symmetric-ish bump, the Poisson’s right skew. These are exactly the shapes your earlier course described with formulas; here they are recovered from samples alone.

Pitfall. A simulated frequency is itself random. With a *small* sample the bars jump around the true pmf, and a law can look wrong when only the sample is small. Before concluding a sampler is broken, enlarge n : genuine agreement should sharpen as the sample grows (the precise rate is the subject of Chapter 3). Judging a law from a few hundred draws is the commonest beginner error here.

2. Simulating the continuous laws

The continuous laws are simulated exactly as the discrete ones were — one `rd` call per law — but the check against theory changes shape. A continuous law has no pmf to match value by value; instead its sample is summarised by a *histogram*, whose outline should trace the law's *density*. This chapter draws the classic continuous laws — uniform, exponential, normal and Pareto — compares each histogram to the density it should match, and ends with the single construction every sampler in the book rests on.

2.1. From samples to a density

```
# The continuous laws are sampled the same way, one call on rd per law.
n = 100000

uniform      = rd.uniform(low=0, high=1, size=n)      # Uniform(0, 1)
exponential  = rd.exponential(scale=1 / 1.5, size=n)  # Exponential(rate = 1.5)
normal       = rd.normal(loc=2.0, scale=0.8, size=n)  # Normal(mu = 2, sigma = 0.8)
pareto       = 1 + rd.pareto(2.5, size=n)             # Pareto(x_min = 1, a = 2.5);
                                                    # rd.pareto starts at 0, the
                                                    # 1 + shifts the minimum to 1

line = "{:<15} sample mean {:7.3f}   theory {:7.3f}"
print(line.format("Uniform(0,1)", np.mean(uniform),    0.5))
print(line.format("Exponential",   np.mean(exponential), 1 / 1.5))
print(line.format("Normal(2,0.8)", np.mean(normal),     2.0))
print(line.format("Pareto(1,2.5)", np.mean(pareto),     2.5 / (2.5 - 1)))
```

Uniform(0,1)	sample mean	0.500	theory	0.500
Exponential	sample mean	0.665	theory	0.667
Normal(2,0.8)	sample mean	1.999	theory	2.000
Pareto(1,2.5)	sample mean	1.667	theory	1.667

The sample means land on their theoretical values, but a mean is a *shape-blind* summary: it fixes the centre and says nothing about the spread or the tail. For a continuous law the sharpest check is visual — overlay the sample's *density histogram* on the theoretical density it should match.

Definition 1 (Density histogram). A *density histogram* of a sample divides the range into bins and draws, over each bin, a bar whose *area* equals the fraction of the sample falling in it. The total area is 1, so the bars are directly comparable to a theoretical density f : for a faithful sample, the bar heights trace the curve $f(x)$.

Turning a histogram into a density one is a single argument, `density=True`; the theoretical curve is then the law's density formula, written out by hand. For the exponential:

```
import matplotlib.pyplot as plt

# A density histogram has bar AREAS summing to 1, so it is directly comparable
```

```
# to a density curve. Here: the exponential sample against its density
# f(x) = 1.5 e^{-1.5 x}, written out by hand.
xs = np.linspace(0, 4, 200)
plt.hist(exponential, bins=60, density=True)      # area-1 bars
plt.plot(xs, 1.5 * np.exp(-1.5 * xs))            # the Exponential(1.5) density
plt.show()
```

The recipe needs each law's density in closed form. Three of the four you already carry: the uniform's is the flat height 1 on $[0, 1]$, the exponential's is the $1.5e^{-1.5x}$ just written out, and the normal's is the bell $f(x) = \frac{1}{\sigma\sqrt{2\pi}}e^{-(x-\mu)^2/(2\sigma^2)}$ of your earlier course. The fourth, the *Pareto*, is new to this course, so we state it in full.

Definition 2 (Pareto law with minimum 1). For a tail index $a > 0$, the *Pareto law with minimum 1* is the continuous law supported on $[1, \infty)$ whose density is

$$f(x) = \begin{cases} ax^{-(a+1)} & \text{for } x \geq 1 \\ 0 & \text{for } x < 1 \end{cases}.$$

Applied to all four laws at once, the recipe gives the comparison below.

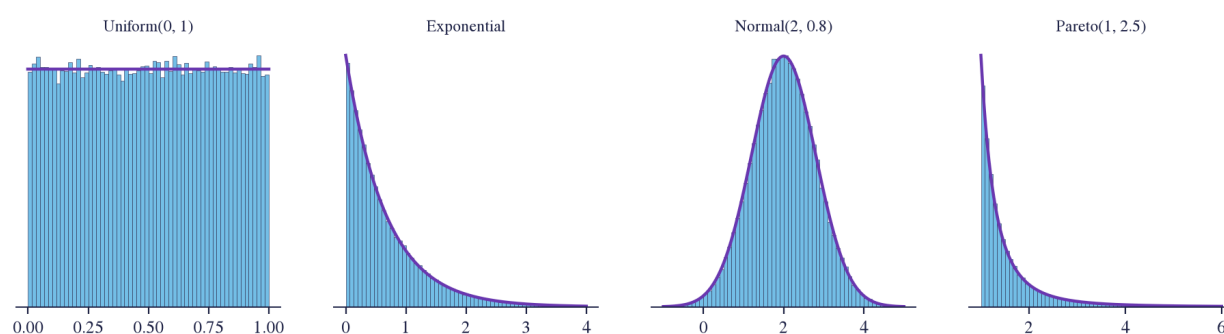


Figure 2: Each classic continuous law, simulated. Bars are density histograms of 100 000 draws; the purple curve is the theoretical density. The *uniform* law is flat on $[0, 1]$; the *exponential* falls off at a constant proportional rate; the *normal* is the familiar symmetric bell; the *Pareto* has a sharp peak at its minimum and a long right tail (shown truncated at 6 — the sample reaches far beyond). Every histogram tracks its density.

Reading Figure 2 fluently is the skill this chapter trains: recognise the flat uniform, the memoryless exponential decay, the symmetric normal bell, and the heavy-tailed Pareto. The last is the one to internalise — a *power-law tail* that a normal-shaped intuition badly under-estimates. Change a parameter or swap in a new law and the same loop redraws the comparison.

Pitfall (The bin count changes the picture). A histogram is not the data — it is a *choice*. Too few bins hide structure (a bimodal sample can look flat); too many make every bar a spike of noise. When a histogram looks surprising, redraw it with a different bin count before drawing a conclusion. The density it *should* match is fixed; the histogram is only an estimate of it.

2.2. Why the tail matters

The exponential and the Pareto both live on $[0, \infty)$ and both slope downward, but they behave very differently. The exponential's tail decays *exponentially* — extreme values are essentially impossible. The Pareto's tail decays only as a *power* of x , so very large values, while rare, appear far more often; the largest of 100 000 draws reaches tens of times above the mean, a spread no exponential sample ever shows. This distinction drives whole fields (insurance, finance, network traffic).

What does such a tail do to our summaries? It is tempting to say “it makes the sample mean unreliable” — but that is only half true, and simulation, the course's own arbiter, says which half. For the Pareto(1, 2.5) used here the tail index 2.5 exceeds 2, so both the mean and the variance are *finite* we can therefore check each by redrawing the same kind of sample many times over and watching how much it moves:

```
one sample : mean = 1.66,  median = 1.32
sample mean   across 400 samples: spread 0.26%
sample variance across 400 samples: spread 92.24%
```

The *sample mean* barely stirs — a spread well under a percent across samples — because a finite variance hands it the ordinary $1/\sqrt{n}$ standard error of Chapter 3. Here it is a perfectly *stable* estimator. What genuinely swings is the *sample variance*: its own variability rests on the *fourth* moment, which the Pareto owns only when the index exceeds 4, so at 2.5 it swings from sample to sample by tens of percent.

The mean is still a poor *typical value* — but for a descriptive reason, not an instability one. The long tail pulls it above the bulk of the data, so it sits above the median (1.32 versus a mean of 1.66 on the sample above). To say where a heavy-tailed sample *sits*, report the median or read the histogram; keep the mean for what it is — the balance point the tail drags rightward.

Remark. Push the tail index down and *two* thresholds matter, not one. Below index 2 the variance becomes *infinite*: the $1/\sqrt{n}$ standard error no longer exists and the $\sqrt{n}$ -shaped Central Limit Theorem fails. The sample mean still converges to the true mean — the Law of Large Numbers asks only for a *finite mean*, which the Pareto keeps for every index above 1 — but it does so more *slowly* and more *erratically* than the $1/\sqrt{n}$ rate. Only when the index drops to 1 or below does the mean itself become infinite and the sample mean *stop settling* altogether, a genuinely inconsistent estimator. The pathology is real, but it arrives in two stages: an unreliable $\sqrt{n}$ error already at index ≤ 2 , and a non-settling mean only at index ≤ 1 .

2.3. Where the samples come from: inverting the CDF

Every `rd` call in this book hides one mechanism. NumPy knows how to make *uniform* numbers on $[0, 1]$; every other continuous law is built from those by a single trick — pass a uniform draw through the *inverse* of the target law's CDF.

Example — an exponential out of a uniform. The exponential law of rate 1.5 has CDF $F(x) = 1 - e^{-1.5x}$. Solving $u = F(x)$ for x inverts it to $x = -\ln(1 - u)/1.5$. So draw $U \sim \text{Uniform}(0, 1)$ and set $X = -\ln(1 - U)/1.5$: an exponential, with no exponential sampler called.

```
# Exponential(rate 1.5) has CDF  $F(x) = 1 - e^{-1.5x}$ , which inverts to
#  $F^{-1}(u) = -\ln(1 - u) / 1.5$ . Feed the `uniform` sample from above through it:
# no exponential sampler is called, yet the result IS an Exponential(1.5).
inv_exp = -np.log(1 - uniform) / 1.5
print("via  $F^{-1}$  of Uniform : mean {:.3f},  $P(X>1)$  {:.3f}".format(
    np.mean(inv_exp), np.mean(inv_exp > 1)))
print("via rd.exponential : mean {:.3f},  $P(X>1)$  {:.3f}".format(
    np.mean(exponential), np.mean(exponential > 1)))
print("theory : mean {:.3f},  $P(X>1)$  {:.3f}".format(
    1 / 1.5, np.exp(-1.5)))
```

```
via  $F^{-1}$  of Uniform : mean 0.667,  $P(X>1)$  0.222
via rd.exponential : mean 0.665,  $P(X>1)$  0.223
theory : mean 0.667,  $P(X>1)$  0.223
```

The sample built by inverting the CDF matches the one `rd.exponential` returns — same mean, same tail probability $\mathbb{P}(X > 1)$, same shape:

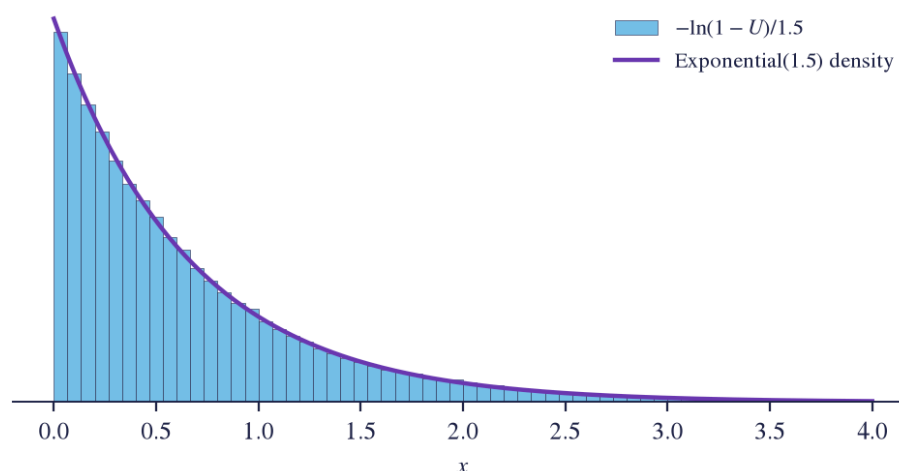


Figure 3: The inverse-transform sample $-\ln(1 - U)/1.5$ (bars), reshaped from `Uniform(0, 1)` draws alone, against the `Exponential(1.5)` density (purple). They coincide — exactly the construction the theorem below justifies.

Why must this work, for *any* law? The claim is a theorem, and its proof is three steps.

Theorem 1 (CDF inversion). Let F be a *continuous, strictly increasing* CDF with inverse F^{-1} , and let $U \sim \text{Uniform}(0, 1)$. Then $X = F^{-1}(U)$ has CDF F :

$$\mathbb{P}(X \leq t) = F(t) \quad \text{for all } t \in \mathbb{R}.$$

Proof. Fix $t \in \mathbb{R}$.

Write the event on X . By the definition of X , namely $X = F^{-1}(U)$, the event $X \leq t$ is the event $F^{-1}(U) \leq t$.

Turn it into an event on U . Because F is strictly increasing it is order-preserving, and so is F^{-1} ; applying F to both sides of $F^{-1}(U) \leq t$, and using $F(F^{-1}(U)) = U$, turns it into $U \leq F(t)$. The two events are therefore the same:

$$\{X \leq t\} = \{U \leq F(t)\}.$$

Read its probability off the uniform CDF. For a $\text{Uniform}(0, 1)$ variable, $\mathbb{P}(U \leq a) = a$ for every $a \in [0, 1]$, and $F(t) \in [0, 1]$ because F is a CDF. Hence

$$\begin{aligned}\mathbb{P}(X \leq t) &= \mathbb{P}(U \leq F(t)) \\ &= F(t),\end{aligned}$$

which says exactly that X has CDF F . □

Remark. This *inverse-transform* method is the engine under `rd.exponential`, `rd.uniform` and the Pareto: a uniform stream, reshaped by an inverse CDF. It needs F^{-1} in closed form — available for those laws, but not the normal, whose CDF has no elementary inverse (NumPy reaches the normal by a different, faster construction).

3. The Law of Large Numbers and the Central Limit Theorem

You met both theorems earlier as statements. This chapter makes them *visible* — and, more importantly, *quantitative* — through simulation. The Law of Large Numbers explains *why* the sample summaries of the previous chapters worked; the Central Limit Theorem tells us *how large the leftover error is* and *what shape it has*, which is the foundation of every confidence interval and test that follows.

3.1. The Law of Large Numbers, watched live

Example — a running average settling down. Flip a biased coin — Bernoulli(0.3) — over and over, and after each flip record the *running average* of all flips so far. Early on the average swings widely; as flips accumulate it settles onto 0.3.

```
# Law of Large Numbers: the running average of Bernoulli(0.3) draws settles on
# the mean p = 0.3. We track the running mean of one long sequence.
draws = rd.binomial(n=1, p=0.3, size=5000)
running = np.cumsum(draws) / np.arange(1, len(draws) + 1)
line = "after {:>4} draws: running mean = {:.3f}"
print(line.format(10, running[9]))
print(line.format(100, running[99]))
print(line.format(1000, running[999]))
print(line.format(5000, running[4999]))
```

```
after   10 draws: running mean = 0.600
after  100 draws: running mean = 0.350
after 1000 draws: running mean = 0.307
after 5000 draws: running mean = 0.306
```

Principle 1 (Law of Large Numbers). For independent draws $X_1, X_2, \dots$ with common mean μ , the running average

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$$

converges to μ as $n \rightarrow \infty$. In words: *the sample mean is a consistent estimator of the expectation* — enough data pins it down.

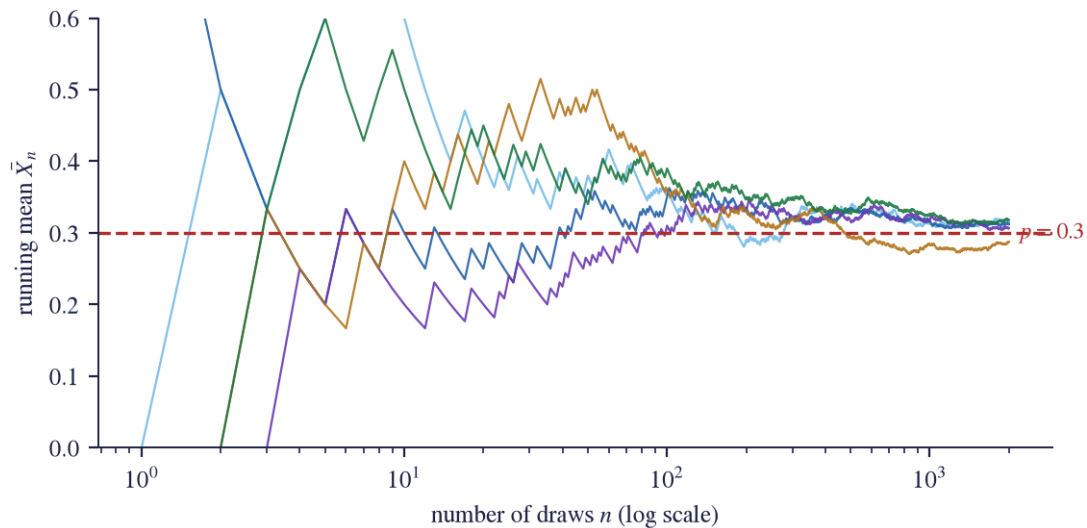


Figure 4: Five independent running averages of Bernoulli(0.3) draws, on a logarithmic n -axis. Each starts erratic and wanders, but all funnel onto the true mean $p = 0.3$ (red dashed) as n grows. The Law of Large Numbers is this funnelling: the spread of where the average can be shrinks steadily toward zero.

Figure 4 also shows what the Law of Large Numbers does *not* promise: at any finite n the average is still off by some amount, and different runs are off by different amounts. Quantifying that leftover error is the Central Limit Theorem's job.

3.2. The Central Limit Theorem, and the size of the error

The Law of Large Numbers says the error $\bar{X}_n - \mu$ vanishes; the Central Limit Theorem says *how fast* and *in what shape*. Its engine is the *standard error*.

Definition 3 (Standard error of the sample mean). If each of the independent draws $X_1, \dots, X_n$ has standard deviation σ , the sample mean $\bar{X}_n$ has standard deviation

$$\text{SE} = \frac{\sigma}{\sqrt{n}}.$$

This standard error is the typical size of the sampling error $\bar{X}_n - \mu$; it shrinks like $1/\sqrt{n}$, so halving the error takes *four times* the data.

Principle 2 (Central Limit Theorem). For independent draws $X_1, \dots, X_n$ with common mean μ and finite standard deviation σ , the *standardised* sample mean

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}}$$

converges in distribution to the standard normal $\mathcal{N}(0, 1)$ as $n \rightarrow \infty$ — *whatever the shape of the original law*. So for large n , $\bar{X}_n$ is approximately normal with mean μ and standard deviation $\sigma/\sqrt{n}$.

The phrase *whatever the shape* is the miracle. We can start from a wildly skewed law and the average still turns normal. Simulation shows it happening.

```
# Central Limit Theorem: the sample mean of n draws from a very skewed law
# (Exponential, rate 1) is itself approximately normal once n is moderate.
# We standardise it, (mean - mu)/(sigma/sqrt(n)), and it should look N(0, 1).
mu, sigma = 1.0, 1.0                                # Exponential(1): mean 1, sd 1
line = "n = {:>2}: mean(z) = {:+.3f}, sd(z) = {:.3f}, skew = {:+.3f}"
for n in [1, 5, 30]:
    means = np.mean(rd.exponential(scale=1.0, size=(20000, n)), axis=1)
    z = (means - mu) / (sigma / np.sqrt(n))
    zc = (z - np.mean(z)) / np.std(z, ddof=1)         # standardise, then 3rd moment
    skew = np.mean(zc ** 3)
    print(line.format(n, np.mean(z), np.std(z, ddof=1), skew))
```

```
n = 1: mean(z) = +0.004, sd(z) = 1.019, skew = +2.126
n = 5: mean(z) = -0.007, sd(z) = 0.999, skew = +0.928
n = 30: mean(z) = -0.003, sd(z) = 0.999, skew = +0.347
```

Read the skew column: the base law (Exponential, at $n = 1$) is strongly right-skewed (skew about 2); by $n = 5$ the skew has more than halved, and by $n = 30$ the standardised average is close to symmetric — the normal shape emerging. The $\text{sd}(z)$ column stays near 1 throughout, confirming the $\sigma/\sqrt{n}$ scaling.

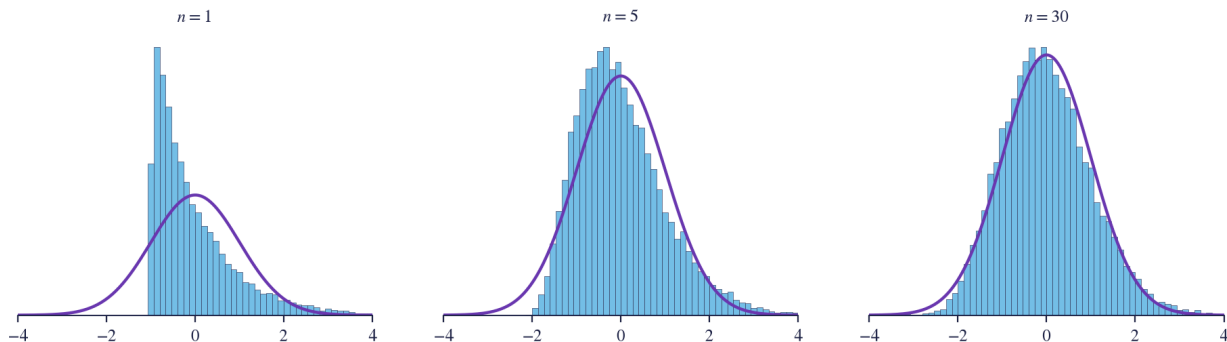


Figure 5: The Central Limit Theorem in action, starting from a strongly skewed law. Each panel is the density histogram of the standardised sample mean of n Exponential(1) draws, against the standard normal (purple). At $n = 1$ it inherits the exponential's skew; by $n = 5$ it is much closer; by $n = 30$ the bell fits well. Convergence is *fast* when the base law is mild, slower for heavy skew.

Example — putting a number on the error. Suppose each observation has $\sigma = 1$ (as for Exponential(1)) and we average $n = 100$ of them. The standard error is $\sigma/\sqrt{n} = 1/\sqrt{100} = 0.100$. By the Central Limit Theorem the sample mean is approximately normal, so about 95% of the time it lands within $1.96 \times 0.100 \approx 0.20$ of the true mean — that interval is precisely a *confidence interval*. The Central Limit Theorem is what licenses reading the error off a single sample.

Pitfall (The Law of Large Numbers is not the Gambler's Fallacy). “The average tends to μ ” does *not* mean a run of high values must be balanced by low ones. Past draws do not push future draws; the average approaches μ only because accumulating *new* independent draws *dilutes* early

deviations, not because the process corrects them. Expecting a “due” outcome after a streak is the Gambler’s Fallacy.

4. Maximum-likelihood estimation

We now turn from *simulating known laws* to *estimating unknown parameters*. You met the maximum-likelihood estimator (MLE) before; here we make its central technique precise — *maximising the log-likelihood* — and carry it out in closed form for the three laws whose estimators you will use most: the Bernoulli, the Poisson and the exponential.

4.1. Likelihood and log-likelihood

Definition 4 (Likelihood and log-likelihood). Let $x_1, \dots, x_n$ be independent data from a law whose density (or pmf) $f(\cdot; \theta)$ depends on an unknown parameter θ ranging over a set Θ . The *likelihood* is the function giving the probability (or density) of exactly that data,

$$\begin{aligned} L : \Theta &\longrightarrow \mathbb{R}_+ \\ \theta &\longmapsto \prod_{i=1}^n f(x_i; \theta) \end{aligned}$$

and the *log-likelihood* is its logarithm $\ell = \ln \circ L$, which turns the product into a sum,

$$\ell(\theta) = \sum_{i=1}^n \ln f(x_i; \theta).$$

Because $\ln$ is increasing, L and ℓ are maximised at the same θ .

Definition 5 (Maximum-likelihood estimator). The *maximum-likelihood estimator* $\hat{\theta}$ is the parameter value that makes the observed data most probable:

$$\hat{\theta} = \operatorname{argmax}_{\theta \in \Theta} \ell(\theta).$$

For a smooth one-parameter model the maximum is found by calculus, and we always work with ℓ , never L : the sum is easier to differentiate than the product, and it avoids the numerical underflow of multiplying thousands of small probabilities. Setting $\ell'(\theta) = 0$ and checking $\ell''(\theta) < 0$ (a maximum, not a minimum) is the whole recipe.

Method 2 (Deriving a closed-form MLE).

1. Write the likelihood and take its logarithm, $\ell(\theta) = \sum_i \ln f(x_i; \theta)$.
2. Solve the first-order condition $\ell'(\theta) = 0$ for θ — this candidate is $\hat{\theta}$.
3. Confirm it is a maximum with the second derivative, $\ell''(\hat{\theta}) < 0$.

4.2. The three closed-form estimators

Example — Bernoulli — the MLE is the sample proportion. Data $x_1, \dots, x_n \in \{0, 1\}$ from Bernoulli(p), with $k = \sum_i x_i$ successes. The likelihood is $L(p) = p^k(1-p)^{n-k}$, so $\ell(p) = k \ln p +$

$(n - k) \ln(1 - p)$. Differentiating, $\ell'(p) = k/p - (n - k)/(1 - p)$; setting this to zero gives $k(1 - p) = (n - k)p$, that is $k = np$, so

$$\hat{p} = \frac{k}{n} = \bar{x}.$$

The estimator is just the *observed proportion of successes* — the intuitive answer, now derived. Here $\ell''(p) = -k/p^2 - (n - k)/(1 - p)^2 < 0$, confirming a maximum.

Example — Poisson — the MLE is the sample mean. Data from $\text{Poisson}(\lambda)$ have $f(x; \lambda) = e^{-\lambda} \lambda^x / x!$, so

$$\begin{aligned} \ell(\lambda) &= \sum_{i=1}^n (-\lambda + x_i \ln \lambda - \ln x_i!) \\ &= -n\lambda + \left(\sum_i x_i \right) \ln \lambda - \sum_i \ln x_i!. \end{aligned}$$

Then $\ell'(\lambda) = -n + (\sum_i x_i)/\lambda = 0$ gives

$$\hat{\lambda} = \frac{\sum_i x_i}{n} = \bar{x},$$

and $\ell''(\lambda) = -(\sum_i x_i)/\lambda^2 < 0$ confirms the maximum.

Example — Exponential — the MLE is the reciprocal mean. Data from the exponential law of rate λ have density $f(x; \lambda) = \lambda e^{-\lambda x}$ on $x > 0$, so $L(\lambda) = \lambda^n e^{-\lambda \sum_i x_i}$ and

$$\ell(\lambda) = n \ln \lambda - \lambda \sum_{i=1}^n x_i.$$

Then $\ell'(\lambda) = n/\lambda - \sum_i x_i = 0$ gives

$$\hat{\lambda} = \frac{n}{\sum_i x_i} = \frac{1}{\bar{x}},$$

and $\ell''(\lambda) = -n/\lambda^2 < 0$ confirms the maximum. Sensible: a fast rate (small mean) means a large $\hat{\lambda}$.

The same three steps — write ℓ and its derivative, solve $\ell' = 0$, confirm with ℓ'' — serve all three laws; only the density changes.

Theorem 2 (Three closed-form MLEs). For a sample $x_1, \dots, x_n$ with mean $\bar{x}$:

$$\hat{p}_{\text{Bern}} = \bar{x}, \quad \hat{\lambda}_{\text{Pois}} = \bar{x}, \quad \hat{\lambda}_{\text{Exp}} = \frac{1}{\bar{x}}.$$

We can confirm all three at once on simulated data, where the true parameter is known:

```
# The three closed-form MLEs, checked on simulated data. In every case the
# estimator is a simple sample statistic derived by maximising the
```

```
# log-likelihood: Bernoulli p -> sample mean, Poisson lambda -> sample mean,
# Exponential rate lambda -> 1 / sample mean.
n = 2000

bern = rd.binomial(n=1, p=0.3, size=n)
print("Bernoulli: true p = 0.30, p_hat = mean(x) = {:.3f}".format(
    np.mean(bern)))

pois = rd.poisson(4.0, size=n)
print("Poisson: true lam = 4.00, lam_hat = mean(x) = {:.3f}".format(
    np.mean(pois)))

expo = rd.exponential(scale=1 / 1.5, size=n) # rate 1.5, so scale 1/1.5
print("Exponential: true lam = 1.50, lam_hat = 1/mean(x) = {:.3f}".format(
    1 / np.mean(expo)))
```

```
Bernoulli: true p = 0.30, p_hat = mean(x) = 0.320
Poisson: true lam = 4.00, lam_hat = mean(x) = 3.878
Exponential: true lam = 1.50, lam_hat = 1/mean(x) = 1.495
```

Each estimate lands beside the truth. The picture behind the algebra is a *hill*: the log-likelihood rises to a single peak at $\hat{\theta}$.

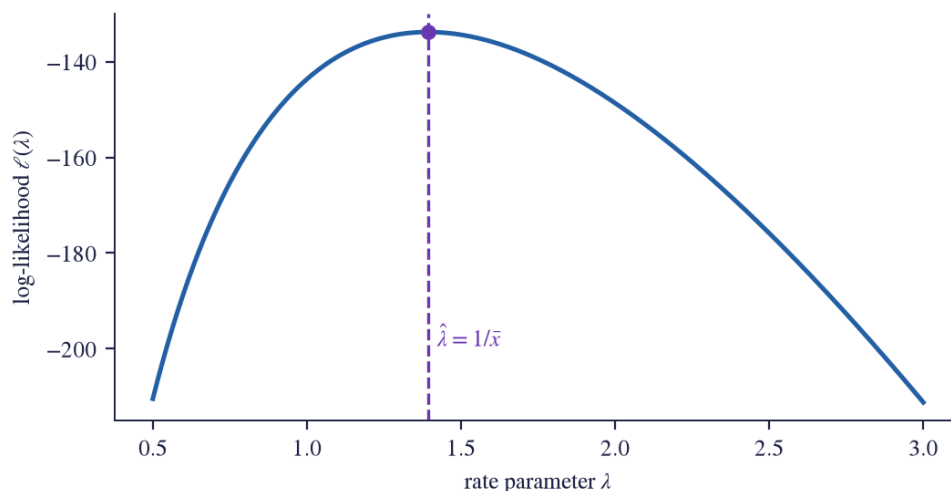


Figure 6: The exponential log-likelihood $\ell(\lambda) = n \ln \lambda - \lambda \sum_i x_i$ for one simulated sample of 200 draws (data drawn at true rate 1.5). It rises to a single peak; the peak sits exactly at $\hat{\lambda} = 1/\bar{x} = 1.392$ for this particular sample (purple dashed) — the MLE of a single finite sample, which need not equal the truth. Setting $\ell'(\lambda) = 0$ is finding the top of this hill.

Remark. Not every law gives a tidy formula. The normal law's two parameters both have closed-form MLEs ($\hat{\mu} = \bar{x}$, $\hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$), but many models — the Pareto's tail index among them — require *numerically* maximising ℓ on a computer. The technique is identical; only the last step changes from algebra to a numerical search.

5. Checking an estimator by simulation

A closed-form estimator is a formula; whether it is any *good* is a separate question. An estimator is a random quantity — feed it a different sample and it returns a different number — so its quality is a property of its *sampling distribution*. This chapter uses simulation to examine three such properties for the exponential MLE $\hat{\lambda} = 1/\bar{x}$: *consistency*, *variance decay*, and *asymptotic normality*. The method is general: it validates *any* estimator whose sampling distribution has no simple formula.

5.1. The repeated-sampling experiment

The idea is to *simulate the estimator's own randomness*. Fix a true parameter, draw many independent samples of size n , compute the estimator on each, and study the resulting cloud of estimates — its centre, its spread, and its shape.

Method 3 (Studying an estimator by repeated sampling).

1. Fix a true parameter value θ and a sample size n .
2. Draw R independent samples of size n ; compute $\hat{\theta}$ on each, giving estimates $\hat{\theta}^{(1)}, \dots, \hat{\theta}^{(R)}$.
3. Their *average* reveals bias (is it centred on θ ?); their *standard deviation* is the estimator's standard error; their *histogram* reveals the shape.
4. Repeat across growing n to see how each property behaves as data accumulate.

```
# Repeated-sampling study of the exponential MLE lam_hat = 1/mean(x), true
# lambda = 1.5. For each sample size we draw 5000 independent samples, compute
# the estimator on each, and summarise the 5000 estimates.
true_lam = 1.5
reps = 5000
head = "{:>6}{:>16}{:>14}{:>14}{:>12}"
row = "{:>6}{:>16.3f}{:>14.3f}{:>14.3f}{:>12.3f}"
print(head.format("n", "mean(lam_hat)", "sd(lam_hat)", "sd * sqrt(n)", "skew(std)"))
mean_by_n, sd_by_n = {}, {}
for n in [10, 40, 160, 640]:
    data = rd.exponential(scale=1 / true_lam, size=(reps, n))
    est = 1 / np.mean(data, axis=1) # one lam_hat per sample
    std = (est - np.mean(est)) / np.std(est, ddof=1) # standardise the estimates
    skew = np.mean(std ** 3) # 0 for a symmetric (normal)
    shape
    mean_by_n[n] = np.mean(est)
    sd_by_n[n] = np.std(est, ddof=1)
    print(row.format(n, np.mean(est), np.std(est, ddof=1),
                    np.std(est, ddof=1) * np.sqrt(n), skew))
```

n	mean(lam_hat)	sd(lam_hat)	sd * sqrt(n)	skew(std)
10	1.673	0.593	1.876	1.481
40	1.537	0.247	1.563	0.624
160	1.511	0.122	1.538	0.328
640	1.502	0.060	1.515	0.104

Every column tells part of one of the three stories.

Definition 6 (Three large-sample properties).

- *Consistency*: $\hat{\theta} \rightarrow \theta$ as $n \rightarrow \infty$ — the estimate homes in on the truth. Here `mean(lam_hat)` marches toward 1.5.
- *Variance decay*: the standard error falls like $1/\sqrt{n}$, so quadrupling n halves it. Here `sd(lam_hat)` roughly halves at each $4 \times$ step, and `sd * sqrt(n)` stays nearly constant — the signature of the $1/\sqrt{n}$ rate.
- *Asymptotic normality*: for large n , the standardised estimator $(\hat{\theta} - \theta)/SE$ is approximately standard normal — the Central Limit Theorem, now applied to the estimator itself. Here `skew(std)`, the skew of the standardised estimates, falls toward 0 — the symmetry a normal shape demands.

Reading the table: quadrupling n from 10 to 40 shrank the spread by a factor of about 2.40 — somewhat above the 2 the $1/\sqrt{n}$ law predicts, an excess left by the small-sample bias that fades as n grows, so that from 160 to 640 the factor is already near 2. The `skew(std)` column falls the same way, from strongly skewed at $n = 10$ to nearly symmetric by $n = 640$: the estimator's own distribution turning normal.

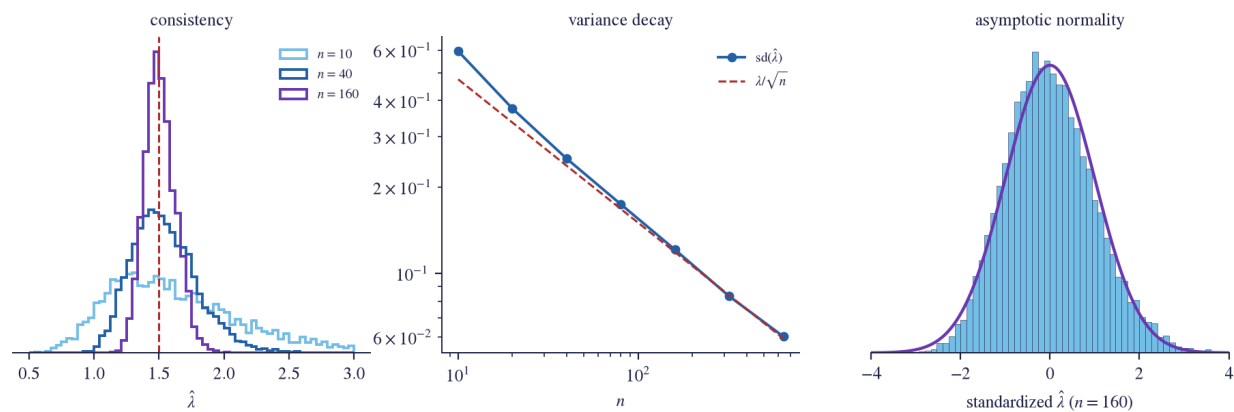


Figure 7: The three properties of the exponential MLE, by simulation (true $\lambda = 1.5$, 5000 samples per size). *Left* — *consistency*: the sampling distribution of $\hat{\lambda}$ narrows around 1.5 (red dashed) as n grows. *Middle* — *variance decay*: on log-log axes, $sd(\hat{\lambda})$ versus n runs parallel to the reference $\lambda/\sqrt{n}$ — a straight line of slope $-1/2$. *Right* — *asymptotic normality*: at $n = 160$ the standardised estimator matches the standard normal (purple).

Pitfall. At *small* n the exponential MLE is visibly *biased upward* — read the table's first row, where `mean(lam_hat)` is 1.673, not 1.5. Consistency is a *large-sample* promise; it does not guarantee unbiasedness at every n . When your sample is small, neither the point estimate nor a normal-based interval around it should be trusted without this caveat.

6. Chi-square tests

The remaining chapters ask the course's defining question directly: *could chance alone have produced what we see?* The chi-square family answers it for *counts*. This chapter covers its two uses: a *goodness-of-fit* test — does a sample of counts match a hypothesised law? — and a *test of independence* — are two categorical variables associated? Both compare *observed* counts to the counts *expected* under a null hypothesis, through one statistic.

6.1. The chi-square statistic

Definition 7 (Chi-square statistic). For observed counts $O_1, \dots, O_k$ in k categories and the counts $E_1, \dots, E_k$ expected under a null hypothesis H_0 ,

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}.$$

Each term measures a category's squared discrepancy, scaled by how many were expected, so a *large* χ^2 signals observed counts far from what H_0 predicts. Under H_0 the statistic follows, approximately, a *chi-square distribution* whose single *degrees-of-freedom* parameter is set by the design; we reject H_0 when χ^2 exceeds the critical value that distribution's table gives for the chosen significance level α . The two tests below differ only in how the expected counts and the degrees of freedom are formed.

6.2. Goodness of fit

Example — is the die fair?. Roll a die 600 times. Under “the die is fair”, each face is expected $E_i = 100$ times. The observed counts differ from 100; the question is whether they differ by *more than chance allows*.

```
# Goodness-of-fit: is a die fair? 600 rolls, expected 100 per face under H0.
# The statistic is computed by hand; the DECISION compares it to a critical
# value read from a chi-square table (given), not to a SciPy p-value.
observed = np.array([89, 110, 96, 113, 92, 100])
expected = np.full(6, np.sum(observed) / 6)
chi2 = np.sum((observed - expected) ** 2 / expected)      # df = 6 - 1 = 5
print("observed = {}".format(observed.tolist()))
print("expected = {}".format(expected.tolist()))
print("chi-square statistic = {:.3f}    (df = 5)".format(chi2))
```

```
observed = [89, 110, 96, 113, 92, 100]
expected = [100.0, 100.0, 100.0, 100.0, 100.0, 100.0]
chi-square statistic = 4.700    (df = 5)
```

The degrees of freedom are $k - 1 = 5$: once five category counts are fixed, the sixth is determined by the fixed total, so only five vary freely. The chi-square table gives, at 5 degrees of freedom and $\alpha =$

0.05, the critical value 11.07. The statistic 4.70 is far below it — counts this uneven arise easily from a fair die — so we *do not reject* fairness.

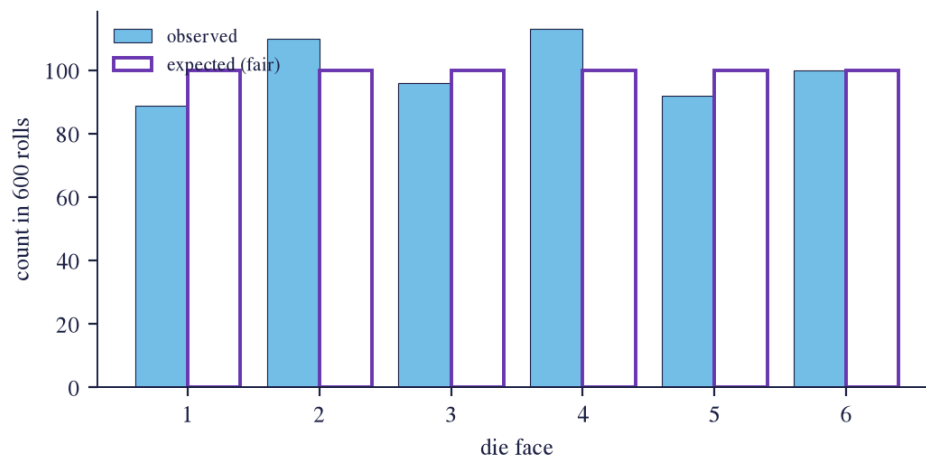


Figure 8: Observed roll counts (blue) against the counts expected from a fair die (purple outline, 100 each). The bars deviate, but the χ^2 statistic of 4.70 on 5 degrees of freedom stays well below the 5% critical value 11.07: deviations this size are unremarkable under fairness, so the data give no reason to call the die loaded.

Method 4 (Chi-square goodness-of-fit test).

1. State H_0 : the law generating the counts (e.g. “each face equally likely”).
2. Compute the expected counts $E_i = n \times \mathbb{P}_{H_0}(\text{category } i)$.
3. Form $\chi^2 = \sum_i (O_i - E_i)^2 / E_i$ with $\text{df} = k - 1 - (\text{estimated parameters})$.
4. Reject H_0 if χ^2 exceeds the table’s critical value at that df and level α .

6.3. Independence of two variables

The same statistic tests whether two *categorical* variables are associated, using a *contingency table* of joint counts.

Definition 8 (Chi-square test of independence). For an $r \times c$ table of observed counts, the expected count under *independence* is

$$E_{ij} = \frac{\text{row}_i \text{ total} \times \text{col}_j \text{ total}}{\text{grand total}},$$

the count if the row and column variables were unrelated. The statistic is

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \quad \text{with} \quad \text{df} = (r - 1)(c - 1).$$

A χ^2 above the table’s critical value at that df is evidence the two variables *are* associated, not independent.

Example — does device predict purchase?. Cross-tabulate purchase (yes/no) against device (mobile/desktop). If the two are independent, the purchase rate is the same on both devices; a large χ^2 says it is not.

```
# Independence: is purchase (yes/no) associated with device (mobile/desktop)?
# The 2x2 contingency table holds observed counts. Expected counts under
# independence are (row total * col total) / grand total; the statistic is the
# same sum (O - E)^2 / E, again decided by hand against a table value.
table = np.array([[90, 60],          # mobile:  bought, not
                  [40, 110]])       # desktop: bought, not
row = table.sum(axis=1, keepdims=True)
col = table.sum(axis=0, keepdims=True)
expected = row * col / table.sum()
chi2 = np.sum((table - expected) ** 2 / expected)      # df = (2-1)(2-1) = 1
print("expected counts under independence:")
print(np.round(expected, 1))
print("chi-square statistic = {:.3f}    (df = 1)".format(chi2))
```

```
expected counts under independence:
[[65. 85.]
 [65. 85.]]
chi-square statistic = 33.937    (df = 1)
```

The code forms the expected counts by the E_{ij} formula above and then sums $(O_{ij} - E_{ij})^2 / E_{ij}$ over the four cells. Here $df = (2 - 1)(2 - 1) = 1$, and the chi-square table gives the critical value 3.84 at level 0.05. The statistic 33.94 is far above it, so the observed table is wildly incompatible with independence: purchase and device *are* associated (mobile buys far more than the independence model predicts).

Pitfall (The approximation needs enough expected count, and shows only association).

The chi-square distribution is only an *approximation* to the true distribution of the statistic, and it degrades when *expected* counts are small — the usual rule is every $E_i \geq 5$. With sparse categories, merge them or use an exact test. And note: a significant χ^2 of independence shows *association*, *not causation* — the same warning that governs any observational cross-tabulation.

7. Comparing a mean or a proportion

Chapter 6 tested counts against a hypothesised law; this chapter tests a *mean* or a *proportion* — against a fixed benchmark, or between two groups. The mechanics (z- and t-tests, significance levels, Type I/II errors) you built earlier; here we apply them to proportions and two-group comparisons, and add the design question those earlier courses left open: *how large a sample do we need?*

7.1. Testing against a benchmark, and between two groups

Definition 9 (z- and t-tests for a mean or proportion). To test whether a parameter differs from a benchmark, form a *test statistic* — (estimate – benchmark) divided by its standard error — and compare it against the distribution it follows under H_0 :

- *One mean vs. a benchmark* μ_0 : $t = (\bar{x} - \mu_0) / (s / \sqrt{n})$, compared to a *Student-t distribution* (use z, the normal, when n is large or σ is known).
- *One proportion vs.* p_0 : $z = (\hat{p} - p_0) / \sqrt{p_0(1 - p_0)/n}$, compared to the normal.
- *Two means (two groups)*: $t = (\bar{x}_B - \bar{x}_A) / \text{SE}$, with the standard error of the difference $\text{SE} = \sqrt{s_A^2/n_A + s_B^2/n_B}$ — each group's own spread, added under the hood — compared to a *Student-t distribution*.
- *Two proportions (two groups)*: $z = (\hat{p}_B - \hat{p}_A) / \text{SE}$ with a pooled standard error, compared to the normal.

A two-sided test at level α rejects H_0 when the statistic exceeds in absolute value the table's critical value — for $\alpha = 0.05$, the normal cutoff is 1.96, and the Student-t cutoff is a little larger and shrinks toward 1.96 as the degrees of freedom grow.

Example — four tests in practice. Four questions, one for each row of the definition above. Does average session time differ from a 30-minute benchmark (one mean, σ unknown, so a t-test)? Does a landing page's click rate differ from a 20% target (one proportion, so a z-test)? Do two page designs differ in average time on page (two means, so a two-sample t-test)? And do two checkout variants convert at different rates (two proportions, so a z-test)?

```
# Three tests, each a statistic computed by hand and then compared to a critical
# value from a table (given): |t| or |z| against 1.96 for a two-sided test at
# level 0.05 (the t cutoff at 49 degrees of freedom is about 2.01).
```

```
# (a) One mean vs a benchmark. Is mean session time different from 30 minutes?
times = rd.normal(loc=32.5, scale=5.0, size=50)      # 50 users, unknown truth
t1 = (np.mean(times) - 30.0) / (np.std(times, ddof=1) / np.sqrt(50))
print("one-sample t      : t = {:.3f}    (mean {:.2f}, n = 50)".format(
    t1, np.mean(times)))
```

```
# (b) One proportion vs a benchmark. Does a landing page's click rate differ
# from a 20% target? The standard error uses the BENCHMARK p0 -- the value H0
# assumes -- not the observed p_hat.
clicks, visitors, p0 = 92, 400, 0.20
p_hat = clicks / visitors
se0 = np.sqrt(p0 * (1 - p0) / visitors)
```

```

z1 = (p_hat - p0) / se0
print("one-proportion z : z = {:.3f}    (rate {:.3f} vs benchmark {:.2f})".format(
    z1, p_hat, p0))

# (c) Two proportions, two groups. Do two checkout variants convert differently?
x_a, n_a = 210, 1000          # control conversions
x_b, n_b = 251, 1000          # treatment conversions
p_a, p_b = x_a / n_a, x_b / n_b
p_pool = (x_a + x_b) / (n_a + n_b)
se = np.sqrt(p_pool * (1 - p_pool) * (1 / n_a + 1 / n_b))
z2 = (p_b - p_a) / se
print("two-proportion z : z = {:.3f}    (rates {:.3f} vs {:.3f})".format(
    z2, p_a, p_b))

```

```

one-sample t      : t = 5.628    (mean 33.78, n = 50)
one-proportion z : z = 1.500    (rate 0.230 vs benchmark 0.20)
two-proportion z : z = 2.177    (rates 0.210 vs 0.251)

```

The one-sample t-test returns $t = 5.628$: with 49 degrees of freedom the 5% cutoff is about 2.01, and 5.628 far exceeds it, so mean session time is convincingly above 30 minutes. The one-proportion z-test returns $z = 1.500$: the landing page's rate 0.230 (92 of 400) sits only 1.500 standard errors from the 20% target — below the 1.96 cutoff, an ordinary distance under H_0 — so the data give no reason to call the rate different from 20%. Note where its standard error comes from: it uses the *benchmark* $p_0 = 0.20$, the rate H_0 assumes, not the observed $\hat{p}$ — the one place this test departs from the two-group form below. The two-proportion z-test returns $z = 2.177$, above 1.96, so variant B's higher conversion (0.251 vs 0.210) is judged real at $\alpha = 0.05$.

That leaves the *two-group mean* comparison — the last of the four rows still unworked. Its standard error is $SE = \sqrt{s_A^2/n_A + s_B^2/n_B}$; dividing the gap in means by it gives the statistic:

```

# (d) Two means, two groups: do two page designs differ in average time on page?
# The standard error of the difference combines each group's own variability,
# SE = sqrt(s_A^2 / n_A + s_B^2 / n_B); the statistic is (xbar_B - xbar_A) / SE.
group_a = rd.normal(loc=30.0, scale=6.0, size=45) # design A, seconds on page
group_b = rd.normal(loc=34.5, scale=6.0, size=45) # design B
se = np.sqrt(np.var(group_a, ddof=1) / len(group_a)
              + np.var(group_b, ddof=1) / len(group_b))
t2 = (np.mean(group_b) - np.mean(group_a)) / se
print("two-sample t : t = {:.3f}    (means {:.2f} vs {:.2f})".format(
    t2, np.mean(group_a), np.mean(group_b)))
print("SE of difference = {:.3f}".format(se))

```

```

two-sample t : t = 2.793    (means 31.17 vs 34.55)
SE of difference = 1.209

```

Here $t = 2.793$, above the two-sided 5% cutoff (about 1.99 at these degrees of freedom), so the two designs differ in mean time on page. The code forms the standard error of the difference by hand and divides the gap by it — the whole test in three lines of NumPy.

7.2. Where the two-proportion statistic comes from

The two-proportion statistic was *quoted* in the code, not derived. Its standard error is worth building from scratch, because every piece is a rule you already have. The derivation is two steps and one admitted fact.

Theorem 3 (The two-proportion z-statistic). Let $\hat{p}_A, \hat{p}_B$ be *independent* sample proportions from groups of sizes n_A, n_B . Under $H_0 : p_A = p_B$, writing $\bar{p}$ for the pooled estimate of the common rate, the statistic

$$Z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\bar{p}(1 - \bar{p})(1/n_A + 1/n_B)}}$$

is approximately standard normal, $\mathcal{N}(0, 1)$.

Proof. Step 1 — the variance of the difference is the sum of the variances. Each proportion $\hat{p} = \frac{1}{n} \sum_{i=1}^n B_i$ averages independent Bernoulli(p) indicators, so $\mathbb{V}(\hat{p}) = p(1 - p)/n$ — the single-draw variance $p(1 - p)$ divided by n , once more the σ^2/n law of Chapter 3. Because the two groups are *independent*, $\text{Cov}(\hat{p}_A, \hat{p}_B) = 0$, so the cross term drops and the variances add:

$$\begin{aligned} \mathbb{V}(\hat{p}_B - \hat{p}_A) &= \mathbb{V}(\hat{p}_A) + \mathbb{V}(\hat{p}_B) \\ &= \frac{p_A(1 - p_A)}{n_A} + \frac{p_B(1 - p_B)}{n_B}. \end{aligned}$$

Step 2 — a linear combination of independent Gaussians is Gaussian. By the Central Limit Theorem each $\hat{p}$ is approximately normal; the difference of two independent normals is normal, so $\hat{p}_B - \hat{p}_A$ is approximately normal. Under H_0 its mean is 0; dividing it by its standard deviation standardises it to $\mathcal{N}(0, 1)$.

The admitted step. Under H_0 both rates equal a common value, estimated by pooling the two groups into one proportion $\bar{p}$. We *admit* — this is the single step we do not prove — that replacing the unknown common rate by $\bar{p}$ leaves the limiting $\mathcal{N}(0, 1)$ distribution unchanged. This turns the variance into $\bar{p}(1 - \bar{p})(1/n_A + 1/n_B)$ and yields the statistic Z above. $\square$

This is exactly the `p_pool`, `se` and `z2` of the code above — the derivation and the three lines of NumPy are one and the same.

7.3. How large a sample?

The single most valuable discipline in comparative testing is fixing the sample size *before* collecting data. Four quantities are locked together — pin three and the fourth follows.

Definition 10 (The four design quantities).

- the *significance level* α — the tolerated false-positive (Type I) rate;
- the *power* $1 - \beta$ — the chance of detecting a real effect of the size you care about;
- the *minimum detectable effect* δ — the smallest difference worth catching, a business choice, not a statistical one;

- the *sample size* n .

Theorem 4 (Sample size for comparing two proportions). To detect a difference δ between two conversion rates, at level α and power $1 - \beta$, the required *per-group* sample size is approximately

$$n \approx (z_{1-\alpha/2} + z_{1-\beta})^2 \cdot \frac{2\bar{p}(1-\bar{p})}{\delta^2},$$

where $\bar{p}$ is the assumed baseline rate and $z_{1-\alpha/2}$, $z_{1-\beta}$ are read from the normal table. The corresponding rule for a difference in *means* replaces $2\bar{p}(1-\bar{p})$ with $2\sigma^2$.

```
# How many users per group to detect a lift from 0.20 to 0.22 at level 0.05
# with power 0.80? The two z-values are read from the normal table (given):
# z_{0.975} = 1.960 and z_{0.80} = 0.842.
z_alpha = 1.960      # z_{1 - alpha/2}, two-sided level 0.05
z_beta = 0.842       # z_{power}, power 0.80
p_bar, delta = 0.20, 0.02
n = (z_alpha + z_beta) ** 2 * 2 * p_bar * (1 - p_bar) / delta ** 2
print("required per-group n = {:.0f}".format(np.ceil(n)))

# Halving the effect we insist on catching quadruples the sample size:
n_half = (z_alpha + z_beta) ** 2 * 2 * p_bar * (1 - p_bar) / (delta / 2) ** 2
print("for delta = 0.01      n = {:.0f}    (4x larger)".format(np.ceil(n_half)))
```

```
required per-group n = 6281
for delta = 0.01      n = 25124    (4x larger)
```

Pitfall (Small effects are dramatically expensive). Because $n \propto 1/\delta^2$, *halving* the effect you insist on catching *quadruples* the sample you need — the run above jumps from about 6281 to 25124 per group. Read the dependence the other way too: a smaller α or a higher power also inflates n . Choosing δ — deciding how small an effect is even worth detecting — is the lever that controls the entire cost of the experiment.

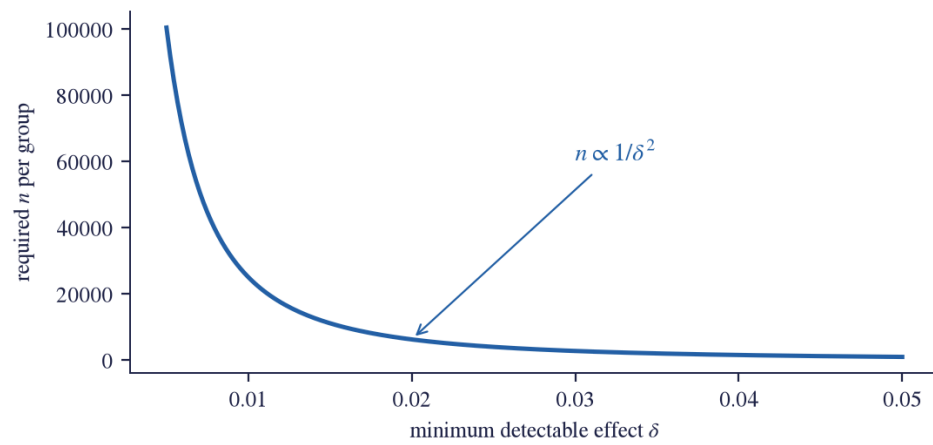


Figure 9: Required per-group sample size against the minimum detectable effect δ (baseline rate 0.20, $\alpha = 0.05$, power 0.80). The curve is $n \propto 1/\delta^2$: it explodes as $\delta \rightarrow 0$. Insisting on catching an effect half as large moves you four times up the curve — the practical reason experiments target the *largest* effect that would still matter.

8. Resampling methods

Every technique so far needed either a formula for a statistic's variability or a known law to simulate from. Two ideas dispense with both, drawing the variability *from the data itself* by reusing the sample in a different way each time. They are the *bootstrap* and the *permutation test*. This chapter is a conceptual tour: what each does, and when each applies.

8.1. The bootstrap: resample with replacement

Principle 3 (The bootstrap idea). When you cannot draw fresh samples from the true population, draw them from the *best stand-in you have* — the observed dataset. Sampling n values *with replacement* from the data, over and over, produces many alternative datasets the world might have given you; computing the statistic on each shows how much it varies. The spread of those values estimates the *standard error* of the statistic, and their percentiles give a *confidence interval*.

Sampling *with replacement* is the whole point: each resample repeats some observed values and omits others, a plausible alternative sample, rather than merely reordering the same values. The bootstrap's reach is that it never inspects *which* statistic it is handed — a median, a 90th percentile, a correlation — so it supplies a standard error and an interval even where no textbook formula exists.

Pitfall (What the bootstrap cannot do). A resample can only contain values the original sample already holds, so the bootstrap cannot manufacture diversity the data lack. It underestimates variability when n is *small*, mis-states it for *heavy-tailed* data whose tail the sample under-represents, and fails outright for the sample *maximum*, whose resampled version can never exceed the observed maximum.

8.2. The permutation test: reshuffle without replacement

Principle 4 (The permutation idea). Under the null hypothesis that two groups are drawn from the *same* distribution, the group labels are *exchangeable*: which observations were labelled “A” and which “B” was arbitrary. So to see what chance alone produces, pool the data and *reshuffle the labels* — reassign them at random, *without replacement*, to the same pooled values — and recompute the difference between the groups. Repeating this builds the *null distribution* of the difference directly from the data; the observed difference is significant when it sits far out in that distribution's tail.

Reshuffling *without* replacement is what makes this a test rather than a measure of uncertainty: it keeps the pooled values fixed and only relabels them, exactly what the null hypothesis says is arbitrary. Its one requirement is *exchangeability under the null* — the pooled observations must be interchangeable if H_0 is true. This holds for a randomised two-group comparison but breaks when the groups differ in some structured way the shuffle destroys (paired data, for instance, are permuted *within* pairs, not across).

8.3. Which to reach for

Framework 1 (Bootstrap vs. permutation).

- The *bootstrap* resamples *with replacement*, each resample the *same size* as the data. It estimates the *variability of an estimator* — a standard error or confidence interval. It answers *how uncertain is my estimate?*
- The *permutation test* reshuffles labels *without replacement*, keeping the pooled values fixed. It builds the *null distribution of a test statistic*. It answers *could chance alone explain this difference?*

Both are Monte Carlo *on the data itself*, and both express the course's guiding habit — before a decision rests on a number, build the distribution of what chance alone produces and hold the observation up against it. The bootstrap builds it for an *estimate* the permutation test builds it for a *difference*.

Primary sources

The applied, simulation-first spirit of this course draws on a small number of accessible books; each is the natural next step for a reader who wants more.

- *Allen B. Downey, Think Stats: Exploratory Data Analysis* (O'Reilly, free online). The closest companion to this course: probability and statistics done in Python through simulation and resampling rather than calculus. Read it alongside the simulation and resampling chapters.
- *David Diez, Mine Çetinkaya-Rundel & Christopher Barr, OpenIntro Statistics* (free). A careful conceptual grounding for populations, distributions, chi-square tests and inference — the reference for the testing chapters.
- *Charles Wheelan, Naked Statistics* (W. W. Norton, 2013). Non-technical intuition for variability, sampling, and why an estimate without its uncertainty misleads — the plain-English counterpart to the discipline this whole course insists on.